ASUR — Project Explainer & Interview Guide

> ***ASUR*** *— a local-first, key-free AI Content Creation OS: one idea in, a researched script, a real rendered video, an honest pre-publish evaluation, and a learning loop that makes the next video better than the last. Repository:* `prathamshukla28/asur` *· License: MIT · Author: Pratham Shukla*

Part 1 — What Is ASUR?

The one-line pitch

Give ASUR a one-line idea. It researches, writes a scored structured script, plans and renders a real 9:16 video from your own machine, honestly checks whether the finished video has a real chance to perform, stops for a human to approve, and after publishing it measures the real result and feeds that back so the next script is informed by what actually happened.

The problem it solves

AI generation and editing are now commodities — nearly every tool resells the same frontier models (Veo, Kling, Runway). The genuinely empty space is elsewhere:

- Honest pre-publish evaluation barely exists. The one tool that ships a

virality score grades the same vendor's own output — a fox guarding the henhouse.

- A creation-to-performance learning loop for creators is essentially missing. A

generated video today has no memory of how the last one actually performed.

- Nobody structurally separates the thing that *makes* the video from the thing

that *grades* it.

ASUR is not another generator. Its defensibility is the closed loop: an honest, firewalled viral evaluator plus cross-video learning memory, running 100% on the local machine with no keys and no network for identity.

Who it's for

Short-form creators who want a disciplined, explainable, local pipeline from idea to Instagram Reel / YouTube Short — and who want each video to learn from the measured performance of the last, instead of guessing.

Part 2 — What It Does

ASUR is a Python package (`asur`) exposing one CLI with several subcommands. The whole stdlib-only core runs with zero third-party packages installed.

```
asur script --idea "why most people waste money on AI tools" --project .
asur run --project .                 # drive the orchestrator forward
asur render --project . --platform youtube_shorts  # real MP4 + upload package
asur explain --project .             # decision trail, read-only
asur verify-repro --project .        # rebuild the chain, compare checksums
asur insights --project .            # what has worked for THIS creator
asur ab-resolve --project .          # measured winner among hook variants
```

`asur script` runs stages 1–7 (idea → research → audience → strategy → hooks → script → a 13-check quality gate). Each stage writes a versioned, checksummed, provenance-bound artifact into the project's `.script/` workspace and prints one line explaining what it produced.

Example run

```
$ asur script --idea "why most people waste money on AI tools" --project .

ASUR SCRIPT - Phase 1 (intelligence)
  project : demo
  root    : /path/demo/.script
  creator : pratham@example.com
  idea    : why most people waste money on AI tools

Building (each line is a versioned, checksummed artifact):
  [ 1] idea      idea-ab12cd34ef56  - understood the idea
  [ 2] research  research-...        - angles, supporting points, risks
  [ 3] audience  audience-...        - pains, desires, objections
  [ 4] strategy  strategy-...        - objective, single promise, CTA
  [ 5] hooks     hooks-...           - 20+ hooks scored, best selected
       hook memory: recorded 20 hooks (append-only)
  [ 6] script    script-...          - structured script + pattern interrupts
  [ 7] qa_report qa_report-...       - 13-check quality gate -> SHIP

Quality gate:
  disposition : SHIP
  passed      : yes

Result: script cleared the quality floor. Ready for GENERATION.
```

Exit code 0 means the script cleared the quality floor; non-zero means it was HELD for revision and nothing was overwritten.

Part 3 — Architecture

ASUR is a staged pipeline where every stage emits exactly one typed artifact into a versioned, append-only store. The pipeline is cut in half by a firewall: GENERATION and VIRAL CHECK never talk to each other; SCRIPT is the only bridge.

The core loop

```
SCRIPT -> GENERATION -> EDITING -> VIRAL CHECK -> PUBLISH -> PERFORMANCE
  ^                                                            |
  +------------------------- LEARNING <-------------------------+
```

The package layout

Package	Responsibility
`asur/core/`	Universal artifact envelope, canonical hashing, gates, local identity, versioned workspaces.
`asur/script/`	Stages 1–7: idea, research, audience, strategy, hooks (+ hook memory), script, 13-check quality gate.
`asur/generation/`	Creative direction, visual screenplay, asset + generation plans, hero proof, QA, render, provenance/license guard, model adapters.
`asur/editing/`	Human-owned timeline, 15 edit primitives, captions, audio mix, edit controls, OTIO adapter.
`asur/viral_check/`	12-dimension honest evaluation reading only the VIRAL_VIEW; verdict PASS / PASS-WITH-CHANGES / REJECT.
`asur/learning/`	Publish package, metrics ingest, performance, A/B resolution, insights, the learning fold.
`asur/orchestrator/`	18-state machine, control tokens, run log, observability.
`asur/platforms/`	Instagram Reels, YouTube Shorts, YouTube Long format packs.
`asur/tools/`	Firewall projection, hash-chained hand-off queue, status, daily driver, explain, verify-repro.
`products/`	Two separate products (Life Mentor, Weekly Tracker) reusing the stdlib core.

The eight founding invariants

Every line of code upholds eight non-negotiable rules. They are the spine of the design:

- `ASUR-LOCAL-01` — 100% local, key-free. Identity is the OS user plus git config only; no network for identity.
- `ASUR-HONEST-01` — never a single "% viral" number; score input quality against documented signals, never a predicted outcome.
- `ASUR-FIREWALL-01` — GENERATION and VIRAL CHECK never communicate; SCRIPT bridges them via two computed projections.
- `ASUR-GATE-01` — fail closed; a missing input, check, or approval caps the disposition at HOLD or BLOCK.
- `ASUR-HUMAN-01` — humans own inputs, approvals, and history; the machine never self-approves silently or publishes.
- `ASUR-PROV-01` — every media asset carries its origin, model, inputs, transforms, and a license block.

- `ASUR-EXPLAIN-01` — every decision is explainable with a recorded reason.
- `ASUR-VERSION-01` — nothing is silently overwritten; artifacts are versioned,

append-only, and checksummed.

The universal envelope

Every stage emits an `Artifact` (schema `asur.artifact/v1`): a typed, versioned, provenance-bound record. The key design choice is that the SHA-256 checksum is computed over the canonical *body* only, not the whole envelope. So re-reviewing or superseding an artifact (which changes envelope metadata) never changes the content hash. The `artifact_id` embeds a short prefix of that body checksum, so identical bodies in the same kind collide intentionally — same content, same id.

Part 4 — The Firewall (the signature subsystem)

The firewall is ASUR's defining mechanism and the hardest thing to get right. The rule: the system that *makes* a video and the system that *grades* it must never see each other's workbench. If the generator can see the checker's pass criteria, it tunes content to whatever the checker rubber-stamps, and the grade becomes meaningless.

Two computed projections

SCRIPT holds full memory and computes exactly two views, in `asur/tools/projection.py`:

- `GENERATION_VIEW` carries the creative brief, hooks, quality/hardness *floors*

(the minimum bar), and the cost/license policy — but never the viral checker's pass criteria, gate thresholds, or finding/weakness/exploit reasoning.

- `VIRAL_VIEW` carries the rendered-bundle content hashes, provenance + per-asset

license facts, platform facts, and rubric metadata — but with the quality and hardness *targets* stripped by name. An evaluator holding the author's playbook grades the intent instead of the bytes.

Three enforcement layers, all fail-closed

- **Pure projection.** Each view is recomputed deterministically from its inputs

— same inputs, same view, no hidden state, no network.

- **Named-forbidden strip + conformance scan.** `conformance_scan` recursively

walks a view — every dict key and every string value at any depth — and raises `FirewallLeak` (mapping to BLOCK / `FIREWALL_LEAK`) if any forbidden token appears, as a casefolded substring match so a name cannot hide inside a larger key or value. `assert_view_clean` selects the forbidden set from the view's own `view` tag, so the correct playbook is always the one withheld, and runs on the exact structure a lane will consume rather than on `repr()`.

- **Static guards in CI and tests.** A CI grep step fails the build if a

forbidden target name appears anywhere in the generation/editing/viral paths, and `tests/test_boundary.py` lints every stdlib module so no cross-import sneaks in.

The hash-chained hand-off queue

The *only* channel from GENERATION to VIRAL CHECK is an append-only JSON-lines queue (`asur/tools/queue.py`) with a seal → claim → verdict → place lifecycle, kept entirely key-free (SHA-256 over canonical bytes plus atomic file ops, no signing). A sealed bundle is frozen: re-sealing the same task with different bytes is refused. A single claimant is guaranteed by an `O_EXCL` lock file. A bundle only advances when the sealed digest, the verdict digest, and the on-disk digest all three agree — so the verdict is provably about the exact bytes on disk.

Part 5 — Honest Provenance, Reproducibility, and the Learning Loop

Three subsystems make ASUR's honesty claims checkable rather than aspirational.

The provenance + license guard

`ASUR-PROV-01` says every asset carries its origin, model, inputs, transforms, and a license block. `asur/generation/provenance.py` makes that real for *every* produced asset, not just the renderer's own output. `validate_produced_assets` fails closed (`ProvenanceError` → BLOCK / `PROVENANCE_INCOMPLETE`) unless each asset carries a non-empty `media_type`, `file_path`, `provenance.model`, `license.name`, and a boolean `license.commercial_ok`. The required shape is deliberately the minimum the pipeline actually relies on — it mirrors exactly what `license_guard` reads — so a validated asset is a license-checkable one, and the check never false-positives on a legitimately produced asset.

verify-repro — reproducibility you can check

`ASUR-VERSION-01` and `ASUR-HONEST-01` claim "same inputs → same checksums". `asur verify-repro` (`asur/tools/verify_repro.py`) turns that into a guarantee. For each stored SCRIPT (and deterministic planning) stage it rebuilds the artifact from its on-disk upstream artifacts, recomputes the body checksum, and compares it to the stored one. A match proves the stage is deterministic and the stored body was not altered out from under its checksum. A mismatch fails closed: it means non-determinism or tampering. Stages that genuinely cannot be rebuilt deterministically from disk (render, viral_check, publish_package, performance, learning) are reported as honestly *skipped* with a reason — never counted toward the verdict. The hooks stage is special-cased because its `prior_hooks` input lives in hook memory and is not recorded in the body; it tries current memory then an empty prior set and records the caveat either way.

The learning fold — proof that learning learns

`asur/learning/learning.py` closes the loop. `build_learning` compares what VIRAL CHECK predicted (the verdict) against what actually happened (measured completion), and records EMPIRICAL learnings — explicitly "this creator's own data, not a universal rule" — plus template-fatigue tracking and a proposed next experiment. `fold_hook_into_memory` then writes the published hook, with its measured result, back into SCRIPT's append-only hook memory, so the *next* `build_hooks` call sees it. A test proves this actually shifts the next run's evaluation of that same hook (fatigue rises, originality drops to derivative) — evidence that the loop learns, not just that the hook is stored. `asur insights` and `asur ab-resolve` surface this: insights aggregates the creator's own hook memory with no aggregate score, and ab-resolve declares an A/B winner only when at least two variants carry a real measured result, failing closed otherwise.

Honest viral evaluation

`asur/viral_check/` scores twelve named dimensions (Attention, Retention, Satisfaction, Value, Shareability, Saveability, Replayability, Originality, Authenticity, Visual Quality, Audience Fit, Platform Fit) each 0–100 on its own, each tagged with an evidence class (verified fact / strong evidence / empirical / hypothesis). The twelve are never collapsed into one number. Penalties are binary risk flags, never percentage deductions; a hard risk flag (watermark, reposted, low resolution) forces REJECT, a soft flag or weak dimension yields PASS-WITH-CHANGES, otherwise PASS. The verdict is deterministic, so the same view always yields the same scores.

Part 6 — Tech Stack

Area	Choice
Language / runtime	Python >= 3.10
Core dependencies	None — the core, SCRIPT, VIRAL CHECK, ORCHESTRATOR, and FIREWALL import only the standard library
Media path (optional extra)	`asur[generation]`: MoviePy/FFmpeg, SDXL (diffusers/torch), Kokoro voice, Stable Audio, faster-whisper, OTIO — pinned, lazy-imported, never on the core import graph
Renderer	FFmpeg/MoviePy is the LOCKED default (zero marginal render cost); Remotion optional only
Identity	OS user (`whoami`) + git config name/email — no keys, no network
Integrity	Canonical JSON + SHA-256 over artifact bodies; hash-chained hand-off queue
Testing	`pytest`; 222 unit tests + 5 opt-in integration tests; no test touches the network or needs a GPU
Quality gates	ruff + mypy (pinned in a `dev` extra), CI on Python 3.10/3.11/3.12, CodeQL, Dependabot
Governance	Frozen SPEC, 9 design docs, ADRs, a constitution (`AGENTS.md`), and a `DEFERRED.md` honesty register

The empty dependency list is a deliberate invariant, not an accident: a stdlib-only install must run SCRIPT, VIRAL CHECK, and orchestration end to end, and CI proves it by snapshotting `sys.modules` before and after import and asserting no forbidden media/network package leaked in.

Part 7 — Honest Limitations

A `DEFERRED.md` register names every obligation the design states but that only a human (or nothing) currently enforces, and a rule leaves it only when something real enforces it:

- The AI *video* adapter (Wan 2.2) is a stub that fails closed; a full idea → real

MP4 works today with procedural colour cards stdlib-only, or with real AI backgrounds + voice/music when the `asur[generation]` extra is installed. The AI image/voice/music/caption adapters are real but human-run, local-only, and never exercised in CI (no GPU, no network).

- Both human gates (hero-proof, publish) are honored in code, but on a solo

machine the producer is the approver, so ASUR records `HOLD:SELF_APPROVED` — an honest state, not a fake pass. A true distinct-approver SHIP needs a second local identity.

- The viral-check dimension scores are a deterministic, explainable heuristic

over documented platform signals — not a trained predictor. That is by design (`ASUR-HONEST-01`): platform ranking weights are unpublished, so ASUR scores signal quality, never predicted reach.

- `verify-repro` can only prove reproducibility for deterministic stdlib stages;

render and human-input stages are honestly skipped, not verified.

- Life Mentor's "encryption at rest" is currently FileVault + strict file

permissions; the data files themselves are plaintext JSON until an opt-in `cryptography` extra lands behind a human gate.

Part 8 — Interview Questions & Answers

A. Project overview

Q1. Give me the 30-second overview of ASUR. ASUR is a local-first, key-free AI Content Creation OS. From one idea it researches, writes a scored structured script, plans and renders a real 9:16 video on your machine, honestly evaluates it on twelve documented dimensions without ever faking a "% viral" number, stops for a human to approve, and after publishing it measures the real result and folds that back into memory so the next video is informed by the last. Its defensibility is the closed loop — an honest, firewalled evaluator plus cross-video learning — not any single model.

Q2. Why did you build it? What gap does it fill? Generation and editing are commodities; everyone resells the same frontier models. The empty space is honest pre-publish evaluation, a creation-to-performance learning loop, and a structural separation between the generator and the evaluator. The one tool that ships a virality score grades its own vendor's output. I wanted a disciplined, explainable pipeline that is honest about what it knows and learns from measured reality.

Q3. Where does the name come from, and what's the core loop? ASUR is the content OS; the loop is SCRIPT → GENERATION → EDITING → VIRAL CHECK → PUBLISH → PERFORMANCE → LEARNING, with LEARNING feeding back into SCRIPT. Every stage produces a versioned artifact with full provenance; nothing is a black box and nothing is silently overwritten.

Q4. What is the current build status honestly? Phase 1 (SCRIPT intelligence) is built and passing — 222 unit tests plus 5 opt-in integration tests. The scripting pipeline, planning, gates, editing plans, FFmpeg render, viral check, publish package, and learning memory are real today. The AI model adapters are a mix of real-but-human-run (SDXL, Kokoro, Stable Audio, faster-whisper) and one stub (Wan 2.2 video). `DEFERRED.md` tracks exactly which refusals and capabilities are real versus intended.

B. Architecture & design

Q5. Walk me through the pipeline. SCRIPT turns an idea into research, audience, strategy, 20+ scored hooks, a structured script, and a 13-check quality gate — seven versioned artifacts. GENERATION turns the script into creative direction, a visual screenplay, asset and generation plans, then produced assets with provenance. EDITING assembles a human-owned timeline. VIRAL CHECK evaluates the rendered bundle on twelve dimensions. A human approves, PERFORMANCE ingests real metrics, and LEARNING folds prediction against reality back into SCRIPT memory. The orchestrator threads state forward and fails closed at the first gate it cannot clear.

Q6. What is the universal envelope and why is the checksum over the body only? Every artifact is a typed, versioned record with a fixed envelope and a stage-specific body. The SHA-256 is computed over the canonical body only, so changing envelope metadata — re-reviewing, superseding, re-filing — never changes the content hash. Content identity is about what was produced, not how it was filed. The artifact_id embeds a prefix of that body hash, so identical bodies collide on purpose, which is what you want for lineage.

Q7. How does the workspace guarantee nothing is overwritten? Versions are monotonic per project and kind. Saving assigns the next version automatically and refuses to clobber an existing version file. Writes are atomic — temp file then `os.replace` — so a crash or a double-run can't corrupt a sealed artifact. Superseding is explicit: a new version is written and the previous latest is marked superseded in a freshly written file, leaving the old file as history. That is `ASUR-VERSION-01` enforced at the persistence layer.

Q8. Why stdlib-only for the core, and how do you keep it that way? `ASUR-LOCAL-01`: the whole control path must run with zero third-party packages, no network, no keys. Media deps live behind the optional `asur[generation]` extra and are lazy-imported, never on the core import graph. I enforce it two ways: a boundary test that parses every core module's imports and asserts they are stdlib or first-party only, and a CI job that installs the package with no extras, snapshots `sys.modules` before and after import, and fails if any forbidden media/network package appeared anywhere in the import graph.

C. Domain

Q9. Why refuse to emit a single "% viral" number? Because a property nobody measured is a property nobody has. Platform ranking weights are unpublished and change over time, and per-video reach is partly serendipitous. A single percentage implies a calibrated prediction that no one can honestly make. So ASUR scores twelve documented signals individually, each tagged with how strong the evidence for that signal actually is, and separates "signal exists" from "weight known". That is `ASUR-HONEST-01`.

Q10. What are the evidence classes and why do they matter? Each dimension and each learning carries one of: verified fact (the platform stated it or it's directly observable), strong evidence (a credible study that transfers only by analogy), empirical (measured from this creator's own past), and hypothesis (plausible creator folklore). They matter because they stop the tool from laundering a guess into a fact — a retention signal the platform confirmed is labelled differently from a replayability hunch.

Q11. What does "fail closed" mean concretely here? A missing input, a failed check, a leaked target name, a digest mismatch, or an incomplete provenance block never passes by default — it caps the disposition at HOLD or BLOCK. `asur run` honestly HOLDs at the first gate it has no checks for rather than faking progress. `asur render` refuses to write a partial package if FFmpeg is missing. Absence of evidence is never treated as a pass.

Q12. What is the ethics/ownership story? Humans own all inputs, approvals, and history. The machine never self-approves silently and never publishes — `asur render` produces files on disk and stops at the publish human gate; a human uploads them. Identity is purely local so it can record who did what on this machine without any remote account.

D. Deep technical

Q13. Explain the firewall and its three enforcement layers. GENERATION and VIRAL CHECK never see each other; SCRIPT computes two projections. GENERATION_VIEW carries quality floors but not the checker's pass criteria; VIRAL_VIEW carries bundle hashes and provenance but has the quality/hardness targets stripped by name. Enforcement is three-layer: pure deterministic projection; a recursive conformance scan that walks every key and string value and raises `FirewallLeak` on any forbidden token (casefolded substring, so it can't hide inside a larger string), selecting the forbidden set from the view's own tag; and static guards — a CI grep over the generation/editing/viral paths plus an import-boundary test.

Q14. How does the hand-off queue prove a verdict is about the right bytes? The queue has a seal → claim → verdict → place lifecycle, key-free, over canonical SHA-256. Sealing freezes a bundle — re-sealing the same task with different bytes is refused. Claiming uses an `O_EXCL` lock file so there is exactly one claimant. Placing only succeeds when the sealed digest, the verdict digest, and the recomputed on-disk digest all three agree; any mismatch is a BLOCK. So the evaluator's verdict provably concerns the exact bytes that exist on disk, with no signing and no network.

Q15. How does verify-repro actually work, and what can't it verify? It rebuilds each stored stage from its on-disk upstream artifacts — the exact inputs the stage ran on — recomputes the body checksum, and compares. A match proves determinism and that the body wasn't altered under its checksum; a mismatch fails closed. It can only verify deterministic stdlib stages (the SCRIPT chain plus the planning stages). Render, viral_check, publish_package, performance, and learning are reported as honestly skipped with a reason and excluded from the verdict — I never claim reproducibility for something I didn't rebuild. Hooks is special-cased because its prior-hooks input lives in memory, not the body.

Q16. How do you make ASUR-PROV-01 real for every asset, not just the render? `validate_produced_assets` walks every asset a production step emitted and fails closed unless each carries a non-empty media_type, file_path, provenance.model, license.name, and a boolean license.commercial_ok. The required shape mirrors exactly what the license guard reads, so a validated asset is guaranteed license-checkable. An undeclared commercial-ok is treated as unsafe and blocks. This deleted a row from `DEFERRED.md` the day it landed.

Q17. How do you prove the learning loop actually learns? Beyond storing a hook, there's a test that publishes a hook, folds it into SCRIPT memory with its measured result, and then runs the hook stage again — and asserts the next run's evaluation of that same hook demonstrably shifts (fatigue rises, originality drops to derivative). That's evidence the memory changes future decisions, not just that data was written. `build_learning` records the prediction-vs-reality diff as EMPIRICAL — this creator's own data, never a universal rule.

Q18. How is reproducibility of CI itself protected? The package is stdlib-only, but the dev tools (ruff, mypy, pytest) and the media extra are pinned to exact versions. Without pinning, a fresh `pip install ruff` in CI pulls whatever released last and can change the rule set out from under the build — which it did. The ruff rule set is also written explicitly in `pyproject.toml` so lint is deterministic across versions and machines.

E. Trade-offs & system design

Q19. Biggest trade-off you made? Keeping the entire control path stdlib-only and key-free. The upside is huge: no network, no accounts, no supply-chain surface for the part that makes decisions, and total reproducibility. The cost is that every integrity mechanism has to be built from SHA-256 and atomic file ops rather than reaching for a signing library, and anything genuinely heavy — the models, FFmpeg — lives behind an optional extra that the core can never import. I enforce that boundary with a test and a CI import probe so it can't erode.

Q20. The viral check is a heuristic, not an ML model. Isn't that weak? It's deliberate. A trained predictor would imply a calibrated probability of reach that no honest system can claim, because the platform weights are secret and reach is partly luck. A deterministic, explainable heuristic over documented signals is something I can defend line by line and that produces the same result every time. The learning loop then grounds it in *this* creator's measured performance, which is the only data that's actually theirs.

Q21. How would you scale this toward real AI video generation? The adapters are already isolated: each model lives in one file under `generation/adapters/`, lazy-imports its heavy deps, fails closed without the extra, and returns a provenance+license block. Wan 2.2 is the one remaining stub; wiring it is adding torch and a GPU-run body behind the same contract, local-only, never in CI. Nothing in the control path changes because the firewall, the queue, and the provenance guard already treat produced assets as opaque bundles with hashes and provenance.

Q22. If a stage became non-deterministic, how would you find out? `verify-repro` would catch it immediately — rebuilding from the same on-disk inputs would produce a different checksum and fail closed. That's the whole point of making reproducibility checkable rather than assumed: a mismatch tells me either the stage drifted or an artifact was tampered with, and both are things a creator must know before trusting the lineage.

F. Behavioral

Q23. What are you most proud of technically? The firewall plus the checkable honesty machinery. The firewall is a genuine structural guarantee — three enforcement layers, all fail-closed — not a comment saying "please don't peek". And verify-repro, the provenance guard, and the closed-loop learning test turn ASUR's honesty claims into things a reviewer can actually run, which is rare. The `DEFERRED.md` register is part of that: a rule can only leave it when real code enforces it, never by being reworded.

Q24. What was the hardest part? Holding the stdlib-only, key-free line while still delivering real integrity. Freezing a bundle, guaranteeing a single claimant, and binding a verdict to exact on-disk bytes — all without a signing library — took the seal/claim/verdict/place queue with `O_EXCL` locks and three-way digest agreement. Getting the firewall conformance scan to catch a forbidden token hidden inside a larger string, at any depth, rather than being fooled by `repr()`, was the other careful piece.

Q25. The repo also hosts Life Mentor and Weekly Tracker — why? They're separate local-first products that reuse four of ASUR's stdlib core primitives (gate, canonical hashing, envelope, evidence) and share its constitution and CI. The dependency direction is one-way and enforced — asur ← life_mentor ← weekly_tracker — and the boundary test lints all three for non-stdlib imports. It keeps one install and zero duplicated code until either product needs its own public life.

Q26. What would you do next? Wire the Wan 2.2 video adapter behind the existing contract, reach a true distinct-approver SHIP with a second local identity, land the opt-in encryption extra for Life Mentor,

and broaden the learning memory. The architecture — stage artifacts, the firewall, the queue, the provenance guard — is built to absorb all of that as local changes behind stable boundaries.

Quick Facts (cheat sheet)

- **What:** a local-first, key-free AI Content Creation OS — idea to rendered

video to honest evaluation to a learning loop.

- **CLI:** `asur script` · `asur run` · `asur render` · `asur explain` ·

`asur verify-repro` · `asur insights` · `asur ab-resolve`.

- **Loop:** SCRIPT → GENERATION → EDITING → VIRAL CHECK → PUBLISH → PERFORMANCE →

LEARNING → back to SCRIPT.

- **Signature feature:** a structural firewall — generator and evaluator never

see each other; SCRIPT bridges them with two conformance-scanned projections.

- **Honesty, made checkable:** no "% viral" number, a per-asset provenance guard,

`verify-repro` byte-for-byte reproduction, and a test proving learning learns.

- **Invariants:** eight non-negotiable rules — local, honest, firewalled, fail

closed, human-owned, provenanced, explainable, versioned.

- **Stack:** Python >= 3.10, stdlib-only core (zero runtime deps), media behind

`asur[generation]`, FFmpeg render, SHA-256 integrity, 222 unit tests, MIT. </content> </invoke>